Shrinking a Shock-Advice Network to Fit an AED Chip

ESP3201A Machine Learning in Robotics and Engineering, Final Project Report
Huang JingWei (A0307445E) and partner · 6 November 2026

Abstract

An automated external defibrillator (AED) has to decide, from a few seconds of ECG, whether a heart in cardiac arrest needs a shock. It makes that call on a small, battery-powered microcontroller. We trained a deep-learning shock-advice model and asked how far it can be compressed, with quantisation and pruning, to fit an AED-class chip (STM32L432: 256 KiB flash, 64 KiB RAM) while still meeting the American Heart Association (AHA) accuracy targets for AEDs. We also asked whether it is better to compress a big model or to train a small one from the start. Using three public PhysioNet databases (23,092 five-second clips from 100 patients) and patient-wise 5-fold cross-validation, int8 quantisation shrank the model about four times at almost no cost, and structured pruning to 35% of the filters brought it to a 93 KiB file needing about 10 KiB of RAM, 21 times smaller than the original. This model meets four of the five AHA targets (coarse VF, rapid VT, normal sinus rhythm and other non-shockable rhythms), one more than the uncompressed model. A network of the same shape trained from scratch did equally well, so pruning gave no advantage over training small. We also found that overall accuracy hides damage to single rhythms: one compressed model kept its AUROC (0.993 to 0.986) while its asystole specificity fell from 60% to 18%. The main caveat is the data: asystole comes from only 6 patients, so no model can show it meets the asystole target with confidence.

1. Introduction

Someone collapses at a void deck. A passer-by opens the AED on the wall, sticks two pads on the chest and presses nothing else. Within about ten seconds the machine has to say one of two things: "shock advised" or "no shock advised". Get it wrong one way and a heart that could have been restarted is left in fibrillation. Get it wrong the other way and a person whose heart is still beating gets an electric shock they did not need.

That decision is made by a rhythm-analysis algorithm running on the AED's microcontroller. Simple enough on paper, but the chip is tiny. The one we target, an STM32L432 (Arm Cortex-M4 at 80 MHz), has 256 KiB of flash to store the program and model, and 64 KiB of RAM to run it. That is smaller than a single photo on a phone. A typical deep-learning model, even a modest one, is measured in megabytes. Our own "large" model is 2 MB in its normal float32 form, eight times too big.

This is where model compression comes in. **Quantisation** stores each number in the model with fewer bits. **Pruning** removes parts of the model that contribute little. Both are standard tools for running neural networks on small devices, but they are usually judged on overall accuracy. An AED is judged differently. The AHA sets a separate target for each type of heart rhythm (Kerber et al., 1997), and a device must meet all of them. A compressed model that keeps its overall accuracy but quietly gets worse at one rhythm would still fail.

So this project asks two questions:

1. **How far can a shock-advice network be compressed to fit an AED-class chip while still meeting the AHA rhythm targets?**
2. **Is it better to compress a big, accurate model, or to train a small model from the start?**

The second question is what gives compression a point. If a small model trained from scratch does just as well, there is no reason to train a big one and shrink it.

We are not the first to put a neural network in an AED. The Avive AED, approved by the US FDA in 2022 (PMA P210015), already uses one. Our contribution is not a new detector. It is a careful look at what compression does to each rhythm category, measured honestly: by patient, with confidence limits, on the target chip's real memory budget.

2. What an AED must get right

So what counts as "right" for an AED? The answer depends on the rhythm.

Shockable rhythms are the ones a shock can fix. In **ventricular fibrillation (VF)** the heart's lower chambers quiver chaotically and pump no blood. In **rapid ventricular tachycardia (VT)** they beat so fast that they barely fill between beats. **Non-shockable rhythms** include normal sinus rhythm (NSR), other abnormal but organised rhythms such as atrial fibrillation (AF), and **asystole**, a flat line, where there is no electrical activity for a shock to reset. Fig. 1 shows a typical example of each.

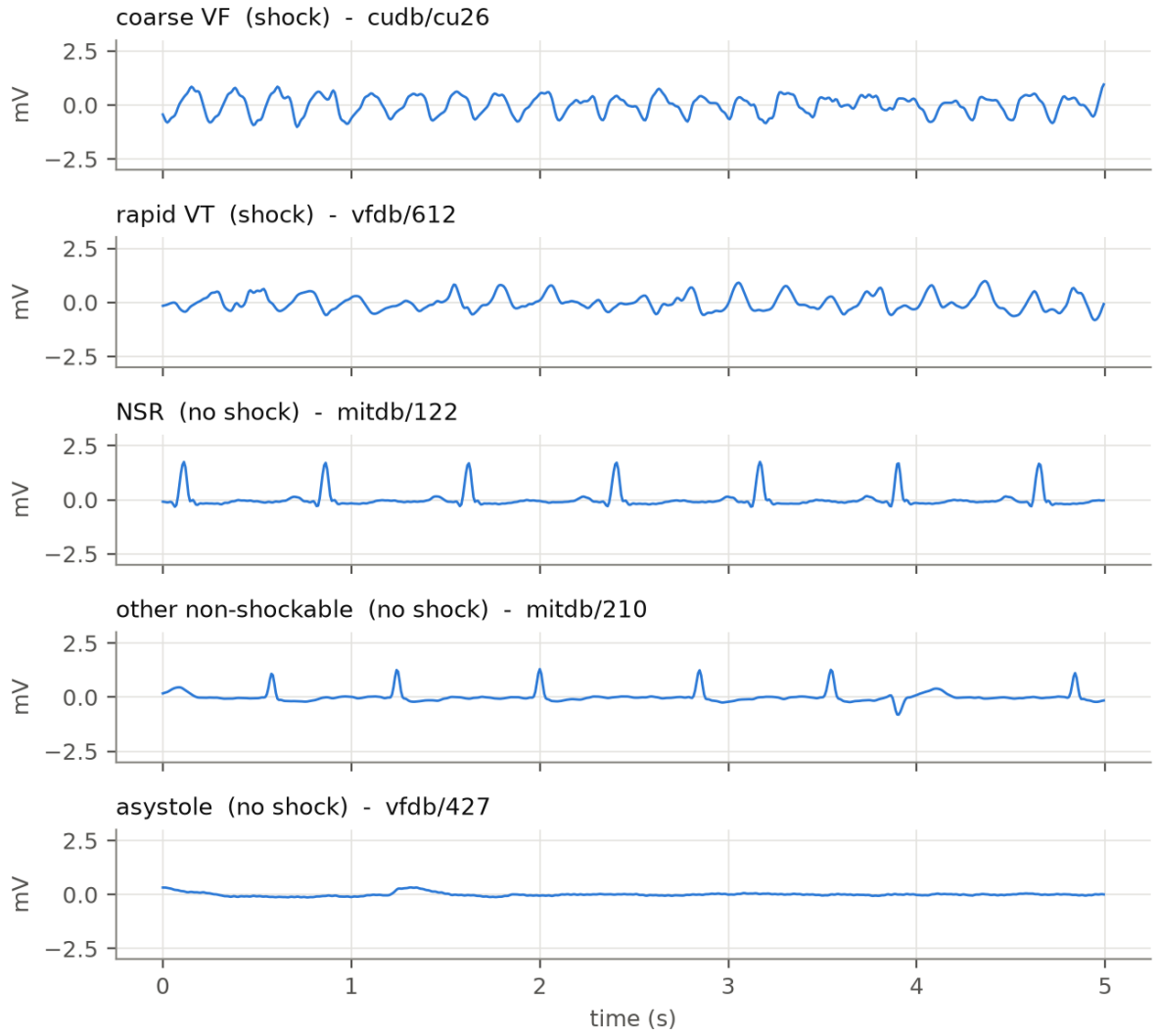


Fig. 1: One typical 5 s clip of each AHA rhythm category (same vertical scale). Coarse VF and rapid VT should be shocked; the other three must not be.

The AHA recommendations (Kerber et al., 1997) turn this into five numbers (Table 1). For shockable rhythms they set a minimum **sensitivity**, the share of shockable clips the device correctly shocks. For non-shockable rhythms they set a minimum **specificity**, the share of non-shockable clips it correctly leaves alone. In other words, sensitivity asks "of the people who needed a shock, how many got one?", and specificity asks "of the people who did not, how many were spared?".

Table 1: AHA performance goals for AED rhythm analysis (Kerber et al., 1997). The last column is the lowest acceptable one-sided 90% lower confidence limit.

Rhythm	Should the AED shock?	Measured by	Goal	Lowest acceptable lower limit
Coarse VF (≥ 200 µV)	Yes	Sensitivity	> 90%	87%
Rapid VT	Yes	Sensitivity	> 75%	67%
Normal sinus rhythm	No	Specificity	> 99%	97%
Other non-shockable (AF, SVT, blocks, ...)	No	Specificity	> 95%	88%
Asystole (< 100 µV)	No	Specificity	> 95%	92%
Fine VF, other VT	Either	Report only	none	none

Two details matter later. First, the NSR target, 99% specificity, is the strictest. Most people an AED is attached to by mistake are fainting, not in cardiac arrest, so false shocks on normal rhythm must be very rare. Second, the AHA asks for a **lower confidence limit**, not just a score. A 96% score measured on 10 patients is far weaker evidence than 96% on 1,000. The lower limit is a pessimistic estimate: the score we can be 90% sure the device beats.

3. How compression works

3.1 Quantisation: fewer bits per number

A trained network is mostly a long list of numbers, its **weights**. Normally each is stored as a 32-bit floating-point number (float32), which can hold values like 0.04127 with about seven digits of precision. **Quantisation** stores each one as an 8-bit integer (int8) instead: a whole number from −128 to 127. That alone makes the model four times smaller.

So how does an integer stand in for 0.04127? Each tensor gets a **scale** and a **zero point**, and

real value ≈ scale × (integer − zero point)

Take one real filter of our large model (Fig. 3a). Its largest weight magnitude is 0.126. TensorFlow Lite gives weights a symmetric range, so the zero point is 0 and the scale is 0.126 / 127 = 0.000993. A weight of 0.03932 becomes round(0.03932 / 0.000993) = round(39.6) = 40. Converted back, 40 × 0.000993 = 0.03970, an error of 0.00038. No weight moves by more than half a step (0.0005). That is the whole trade: a small rounding error on every number, in exchange for a four-times smaller model and integer-only arithmetic.

The weights are only half the story. While the model runs, every layer also produces **activations**, the intermediate signals passed to the next layer. There are two common ways to handle them:

- **Dynamic-range quantisation** stores the weights as int8 but keeps activations in float32. The weights are converted back to float on the fly. It needs no data and shrinks the file about four times, but the chip still does floating-point maths.
- **Full-integer quantisation** makes activations int8 too. Their ranges are not known in advance, so the converter runs a few hundred **calibration** clips through the model and records how large each activation gets. That range sets each activation's scale and zero point. The result runs entirely on integer arithmetic, which is what fast microcontroller kernels such as CMSIS-NN are built for (Lai et al., 2018).

Both are **post-training quantisation (PTQ)**: the model is trained normally and converted afterwards (Jacob et al., 2018; Krishnamoorthi, 2018). The alternative, **quantisation-aware training (QAT)**, simulates the rounding during training so the model learns to tolerate it. QAT is only worth its extra cost when PTQ loses accuracy. Here PTQ cost very little overall accuracy (Section 6.2), so we did not use it, although Section 7 suggests where it could still help.

3.2 Pruning: removing parts of the model

Pruning removes the parts of a network that matter least. The common way to judge "matters least" is size: a weight close to zero barely changes the output, so it can go. There are two very different ways to remove things.

Unstructured pruning sets individual weights to zero, wherever they are (Han et al., 2015). It can remove a large share of weights with little accuracy loss. The catch is that the network keeps its shape. A layer with 75% of its weights set to zero is still stored and computed as a full layer, unless the hardware and software have special sparse kernels, which a Cortex-M4 running TensorFlow Lite Micro does not. The zeros make the file compress well on disk, but on the chip the model is exactly as big and as slow as before.

Structured pruning removes whole **filters** (the feature detectors in a convolutional layer) instead. Removing filter *j* of a layer removes output channel *j*, and therefore also input channel *j* of the next layer. The result is a genuinely narrower network with fewer weights, less computation and smaller activations. We rank filters by their **L1 norm**, the sum of the absolute values of their weights, and keep the strongest (Li et al., 2017). Fig. 4 shows this for one layer.

There is one complication in our model. It is a residual network (He et al., 2016): each block adds its input to its output through a shortcut. Two tensors can only be added if their channels line up, so the filters on both sides of each addition must be chosen together. We sum their L1 scores and keep one shared set of channel positions for both.

Cutting filters damages the model, so pruning is always followed by **fine-tuning**: a few more epochs of training at a lower learning rate, which lets the remaining filters adjust to the loss of their neighbours. This raises a fair question, asked by Liu et al. (2019): if the pruned network is retrained anyway, why not just train a network of that narrow shape from scratch? We test exactly that.

4. Related work

Deep learning for AED shock advice is well studied. Nguyen et al. (2018) combined a convolutional feature extractor with a boosting classifier on the same PhysioNet databases we use. Jaureguibeitia et al. (2020) showed that convolutional and residual networks can meet the AHA targets on segments as short as a few seconds. These papers report accuracy but not embedded cost: model size, memory or speed on a real chip.

On the other side, the TinyML Design Contest 2022 (Jia et al., 2023) asked teams to detect life-threatening ventricular arrhythmias on the same STM32L432 board we target. Its signals, however, come from inside the heart, recorded by implantable defibrillators, not from AED pads on the chest, and its scoring is not per AHA category.

The gap we fill is the overlap. We found no work reporting how quantisation and pruning change each AHA rhythm category separately, under the memory budget of an AED-class microcontroller.

5. Method

The whole pipeline fits in four words: train, prune, retrain, convert. The rest of this section explains each step and the tools used (Fig. 2).

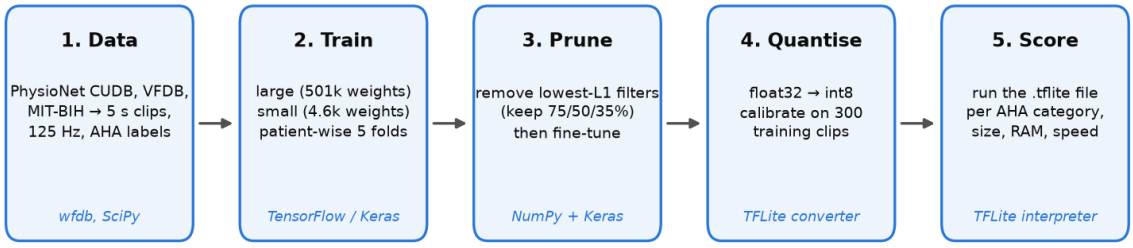


Fig. 2: The pipeline. Every step from training to the final int8 file uses TensorFlow / Keras and the TensorFlow Lite converter; all scoring is done on the .tflite file itself.

5.1 Data

We used three free PhysioNet databases (Goldberger et al., 2000): the Creighton University Ventricular Tachyarrhythmia Database (CUDB), the MIT-BIH Malignant Ventricular Arrhythmia Database (VFDB), and the MIT-BIH Arrhythmia Database (Moody and Mark, 2001). They are the standard sources in shock-advice research, which keeps our results comparable with published work.

Each recording was reduced to one lead, resampled to 125 Hz and band-pass filtered (0.5–30 Hz) to remove baseline drift and muscle noise. It was then cut into non-overlapping **5-second clips** (625 samples), a typical AED analysis window. Each clip takes its label from the databases' rhythm annotations; clips where the rhythm changes, or that are marked as noise, were dropped. VF clips were split into coarse and fine by amplitude (200 µV), and VT clips into rapid and other by heart rate (180 bpm). Asystole clips are taken as annotated. The rate cut-off is our own choice: the AHA statement does not fix one number, so we flag it as an assumption. Table 2 shows the result.

Table 2: Clips and patients per AHA category.

Category	CUDB	VFDB	MIT-BIH	Clips	Patients
coarse VF	713	532	24	1,269	42
rapid VT	2	440	0	442	16
NSR	96	3,329	12,165	15,590	57
other non-shockable	73	2,174	2,838	5,085	36
asystole	0	147	0	147	6
fine VF	1	1	0	2	2
other VT	0	540	17	557	16
Total	885	7,163	15,044	23,092	100

The numbers reveal the biggest weakness of public AED data: NSR has 15,590 clips, but asystole has only 147 clips, all from 6 patients. We return to this in Section 7.

5.2 Splitting by patient

Clips from the same person look alike. If one patient's clips land in both the training and the test set, the model can recognise the person rather than the rhythm, and the test score is inflated. Many published shock-advice results split clips at random and suffer from exactly this.

We split by **patient** instead, with 5-fold cross-validation: the patients are divided into five groups, and each group takes one turn as the test set while the model trains on the other four. Of the training patients, one in six are held back as a **validation** set, used only for early stopping and for choosing a decision

threshold (Section 5.5). In other words, every test score in this report comes from people the model never saw during training or tuning. Fine VF and slow VT clips are never trained on, since the AHA sets them no target, but they are still scored and reported.

5.3 Two models

The **large model** is a one-dimensional residual network: a first convolution, four residual blocks of two convolutions each (32, 64, 96 and 128 filters), and a small dense head. It has 501,537 weights and needs about 36.7 million multiply-accumulate operations (MACs) per clip. It is our "accurate reference": it does not fit the chip.

The **small model** has three plain convolutional layers (8, 16 and 32 filters) and 4,625 weights, about a hundred times fewer. It was designed to fit the chip from the start.

Both read one 5 s clip and output a probability that the rhythm is shockable. Both were trained with the Adam optimiser and binary cross-entropy loss, for up to 40 epochs with early stopping on the validation loss. Because shockable clips are much rarer than non-shockable ones, the two classes were weighted equally during training.

5.4 Compression

From each fold's trained large model, we made the following variants:

- **Structured pruning** keeping 75%, 50% and 35% of the filters in every layer. Because a layer's weights scale with both its input and output width, keeping 50% of filters leaves about a quarter of the weights. Each pruned model was fine-tuned for up to 20 epochs at a learning rate of 3×10^{-4}.
- **Train-from-scratch twins** of the 50% and 35% shapes: the same narrow architecture, randomly initialised and trained like the large model. This is the control for question 2.
- **Unstructured pruning** setting 75% of the weights in every layer to zero, matching the share of weights removed by 50% structured pruning, then fine-tuned with the zeros held at zero.

Every model, compressed or not, was then converted by the TensorFlow Lite converter into three files: float32 (no compression, as the reference), dynamic-range int8 (for the small and large models), and full-integer int8, calibrated on 300 clips drawn from that fold's training patients. All scores in Section 6 are computed by running the **.tflite files themselves** in the TensorFlow Lite interpreter, the same bytes that would be flashed to the chip.

5.5 Measuring success

Accuracy. For every AHA category we compute the score (sensitivity or specificity) on the pooled test predictions of all five folds, and its one-sided 90% lower confidence limit. The limit comes from a bootstrap over patients: we resample whole patients with replacement 1,000 times and take the 10th percentile. Resampling patients rather than clips is stricter, because it accounts for clips from one person being alike. A row **passes** only if the score beats the AHA goal *and* the lower limit reaches the AHA's lowest acceptable limit (Table 1).

Threshold. A model outputs a probability; the AED shocks if it is above a threshold. The usual default is 0.5, but nothing makes 0.5 special. For each model and fold we also chose the threshold on the *validation*

patients: the one with the best worst-case margin over the AHA goals. We report results at both thresholds. The test patients are never used to choose anything.

Fit on the chip. Flash use is the size of the .tflite file. RAM use is estimated as the largest set of activation tensors alive at the same moment while the model runs, a lower bound, as the runtime adds a few KiB on top. A model fits if its file is under 256 KiB and its peak activations are under 64 KiB.

Speed. We did not have a physical board, so on-chip latency is an *estimate*: the MAC count divided by an assumed 0.5–1 int8 MAC per clock cycle with CMSIS-NN kernels, at 80 MHz. We also report the measured time per clip on a laptop CPU.

6. Results

All numbers below are pooled over the five test folds (23,092 clips, 100 patients) and use the 0.5 threshold unless stated otherwise. Section 6.6 explains why the tuned threshold is not the headline.

6.1 The starting point

Before compressing anything, how good are the two uncompressed models? The large float32 model passes 3 of the 5 AHA rows. Coarse VF (94.0%), rapid VT (95.9%) and other non-shockable rhythms (99.4%) clear their goals comfortably. NSR specificity is 98.6%, just under the 99% goal. Asystole is 67.3%, far below 95%. The small model also passes 3 of 5, but its NSR specificity is only 95.6%: about one normal-rhythm clip in 23 would be shocked.

So even the uncompressed model is not perfect. That matters for how we read everything else: the question is not "does compression keep a perfect model perfect", but "does compression make things worse, and where".

6.2 Quantisation: four times smaller, almost free

Table 3: The large model in three file formats (same trained weights).

File	Size (KiB)	AUROC	Decisions changed vs float32 (of 23,092)	AHA rows passed
float32	1,975	0.9937	0	3 / 5
Dynamic-range int8	525	0.9937	17	3 / 5
Full-integer int8	538	0.9889	299	3 / 5

Table 3 shows the trade. Both int8 formats shrink the file about 3.7 times. Dynamic-range quantisation, which rounds only the weights, changes just 17 of 23,092 decisions. Full-integer quantisation, which also rounds every activation, changes 299 (1.3%) and lowers the AUROC slightly, from 0.994 to 0.989. Fig. 3b shows where the changes happen. For 95% of clips the two outputs differ by less than 0.05; most clips sit in the corners, where both versions are confidently right. The damage is concentrated in the 3–4% of clips the model was already unsure about (float32 output between 0.1 and 0.9). Their int8 outputs scatter widely, and the horizontal bands show the output itself snapping to a limited set of levels. Only the uncertain clips that cross 0.5 change the decision. Full-integer int8 also cuts peak activation memory four times, from 117 KiB to 29 KiB, because every activation is now one byte instead of four.

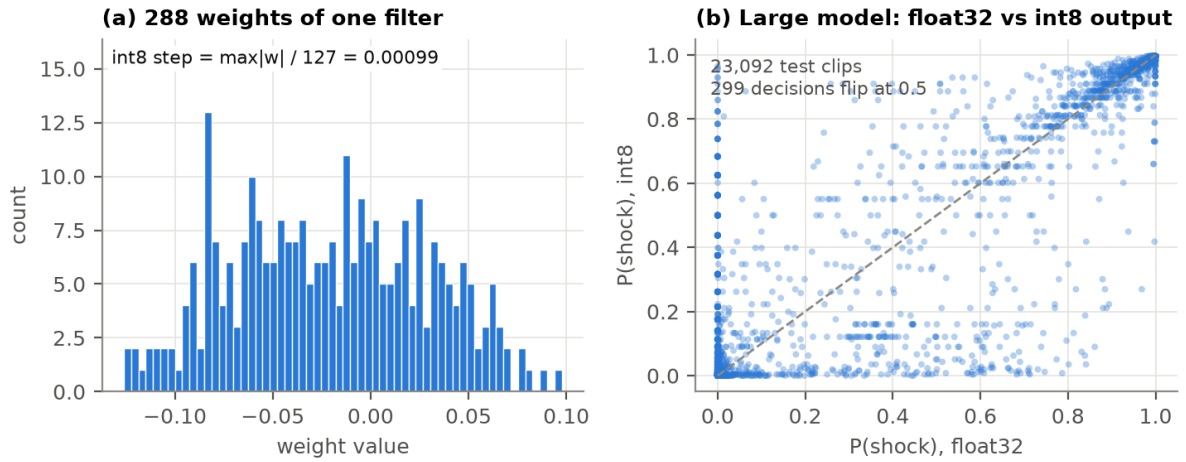


Fig. 3: (a) The 288 weights of one filter in the large model. Full-integer int8 rounds each to the nearest step of 0.001. (b) P(shock) from the float32 and int8 versions of the large model for every test clip; points on the dashed line are unchanged.

But quantisation alone is not enough. At 538 KiB, the int8 large model is still more than twice the chip's 256 KiB of flash. To fit, the network itself has to get smaller.

6.3 Pruning: cut, then repair

What happens straight after filters are cut? Fig. 5 answers it. Removing a quarter of the filters drops the AUROC from 0.99 to 0.90. Removing half drops it to 0.69, not far above a coin toss. Fine-tuning then brings every pruned model back to about 0.99. In other words, the L1 ranking chooses *which* filters to keep, but it is the retraining that makes the smaller network work again. Fig. 4 hints at why the choice matters less than expected: in this layer the L1 norms are nearly flat, so the "weakest" filters are barely weaker than the strongest.

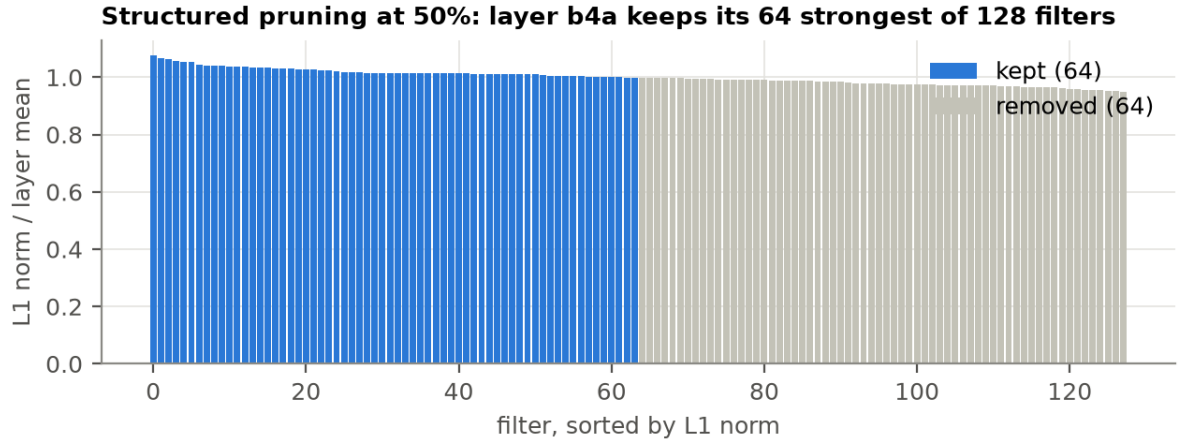


Fig. 4: Structured pruning in one layer (b4a). Its 128 filters are sorted by L1 norm; at 50% pruning, the 64 strongest are kept.

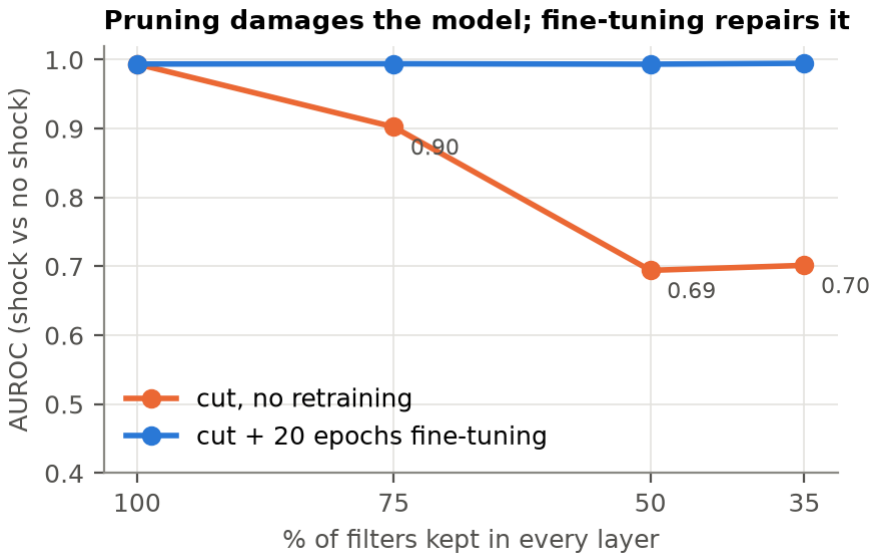


Fig. 5: AUROC of the large model after structured pruning, before (orange) and after (blue) fine-tuning.

The type of pruning matters as much as the amount (Fig. 6). Setting 75% of the weights to zero (unstructured) kept the AUROC at 0.992, but the int8 file stayed at exactly 538 KiB: the zeros are still stored and still multiplied. They only help when the file is gzipped (211 KiB), and a microcontroller cannot run a gzipped model. Removing half the filters (structured), which removes about the same 75% of weights, gives a genuinely smaller 160 KiB file with four times fewer operations.

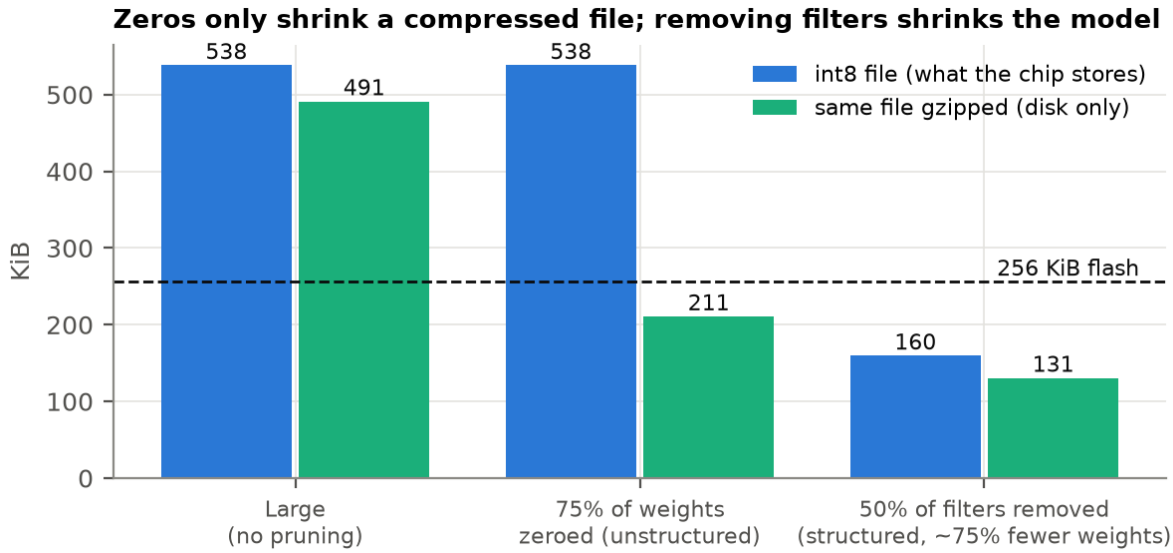


Fig. 6: Int8 file size of the large model, with 75% of its weights zeroed (unstructured) and with half its filters removed (structured). Only the structured model is smaller on the chip.

6.4 What fits, and does it still meet the targets?

Table 4 brings everything together. All models are int8 except the reference in the first row. A ✗ marks a failed AHA row (score below the goal, or lower confidence limit below the AHA minimum).

Table 4: Every int8 model at threshold 0.5. Scores in %. Flash = .tflite file size; RAM = peak activation memory. Chip budget: 256 KiB flash, 64 KiB RAM.

Model	Weights	Flash (KiB)	RAM (KiB)	Fits?	Coarse VF	Rapid VT	NSR	Other non-shock.	Asystole	Passed
Large, float32 (reference)	501,537	1,975	117.4	No	94.0	95.9	98.6 ✗	99.4	67.3 ✗	3 / 5
Large	501,537	538	29.3	No	94.4	96.2	98.2 ✗	98.8	90.5 ✗	3 / 5
Pruned to 75%	282,649	319	22.0	No	91.9	96.2	97.7 ✗	98.2	92.5 ✗	3 / 5
Pruned to 50%	126,097	160	14.7	Yes	87.6 ✗	94.3	99.2	99.3	18.4 ✗	3 / 5
Pruned to 35%	62,347	93	10.1	Yes	91.1	95.7	99.3	99.2	85.7 ✗	4 / 5
50% width, from scratch	126,097	160	14.7	Yes	93.3	93.2	98.4 ✗	98.4	85.0 ✗	3 / 5
35% width, from scratch	62,347	93	10.1	Yes	97.4	96.4	99.2	99.4	61.2 ✗	4 / 5
Small	4,625	13	4.8	Yes	96.1	93.2	95.6 ✗	97.8	82.3 ✗	3 / 5
75% of weights zeroed	501,537	538	29.3	No	94.8	95.2	98.0 ✗	98.3	93.2 ✗	3 / 5

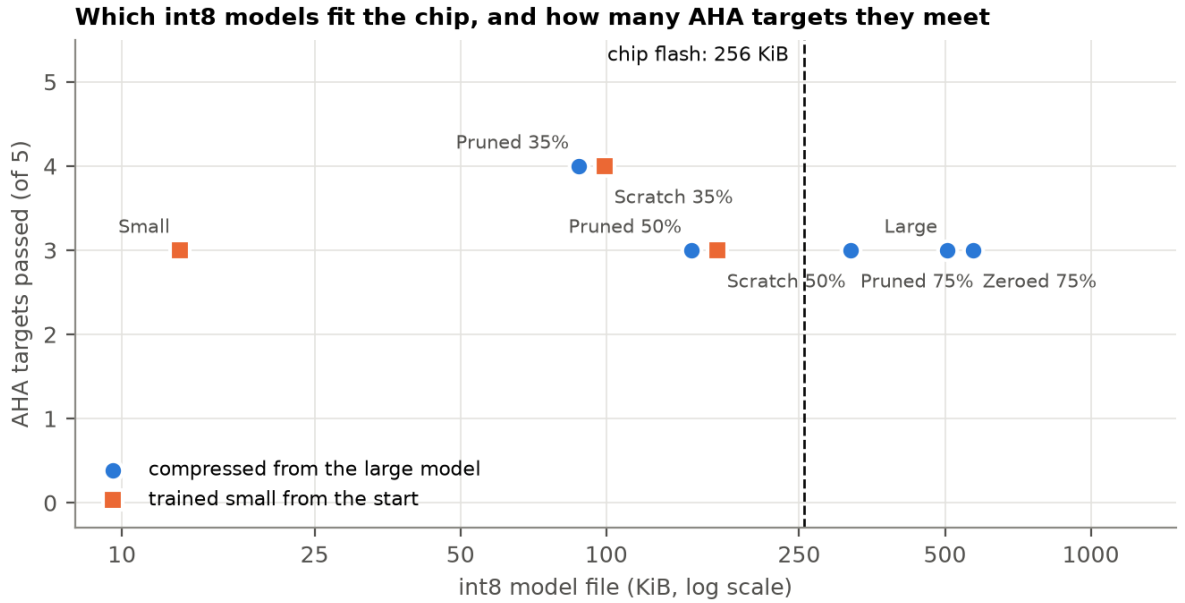


Fig. 7: Int8 file size against the number of AHA targets met. Models left of the dashed line fit the chip's flash. Points sharing a file size are nudged apart so both are visible.

The best models are the two at **35% width**, whether pruned or trained from scratch. Each is a 93 KiB int8 file needing about 10 KiB of activation memory, 21 times smaller than the float32 large model, and each passes 4 of the 5 AHA rows: every row except asystole. That is one more row than the uncompressed large model, because both clear the 99% NSR goal that the large model narrowly misses. Their estimated time per clip on the Cortex-M4 is 56–112 ms, against a 5-second analysis window. Fig. 8 shows their scores category by category.

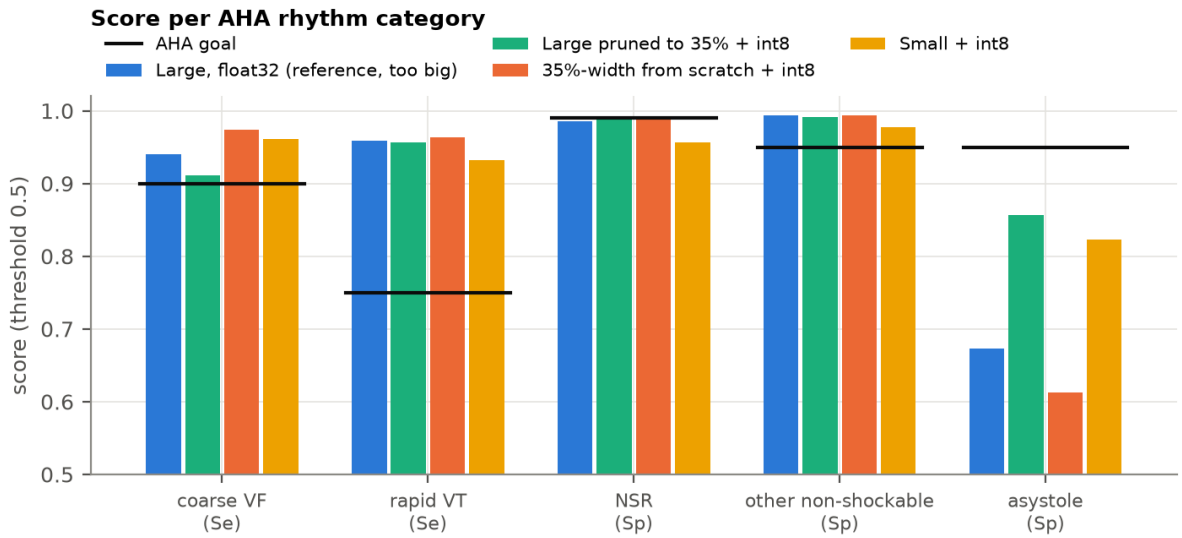


Fig. 8: Score per AHA category for the large float32 reference and three int8 models that fit the chip. Black bars are the AHA goals.

6.5 Compress big, or train small?

This was our second question, and the answer is surprisingly plain. At the same shape, pruning the large model did **not** beat training from scratch. At 35% width both pass 4 of 5 rows, and the scratch model has the wider margins on the shockable rhythms (coarse VF 97.4% vs 91.1%). At 50% width both pass 3 of 5, failing different rows. This matches Liu et al. (2019): what a big model mostly gives you is a good *architecture*, not a set of weights that cannot be reached any other way.

"Train small" has a limit, though. The 4,625-weight small model fits easily (13 KiB) but fails NSR at 95.6%. It is simply too small to tell all normal rhythms from fibrillation. The useful size lies between the two original designs, and the large model is what told us where.

6.6 Which rhythm breaks first?

Asystole, every time, and in an unsettling way. Look at the 50% pruned model. Converting it from float32 to int8 barely changed its AUROC (0.993 to 0.986), the kind of number a compression paper would call "no loss". But its asystole specificity fell from 59.9% to 18.4%: four out of five flat-line clips would now be shocked. An overall metric hid a collapse in one category entirely.

Part of the reason is in the input. Full-integer int8 rounds the incoming ECG in steps of 39 µV, set by the calibration clips. Our other rhythms have a median amplitude of 1,300–1,800 µV, about 35–45 steps. Asystole has a median of 193 µV, about five steps, so the rounding reshapes it far more (Fig. 9). Asystole scores also swing widely between models that differ only slightly (from 18% to 93% across Table 4), which says the models were never confident about it in the first place. We come back to why in Section 7.

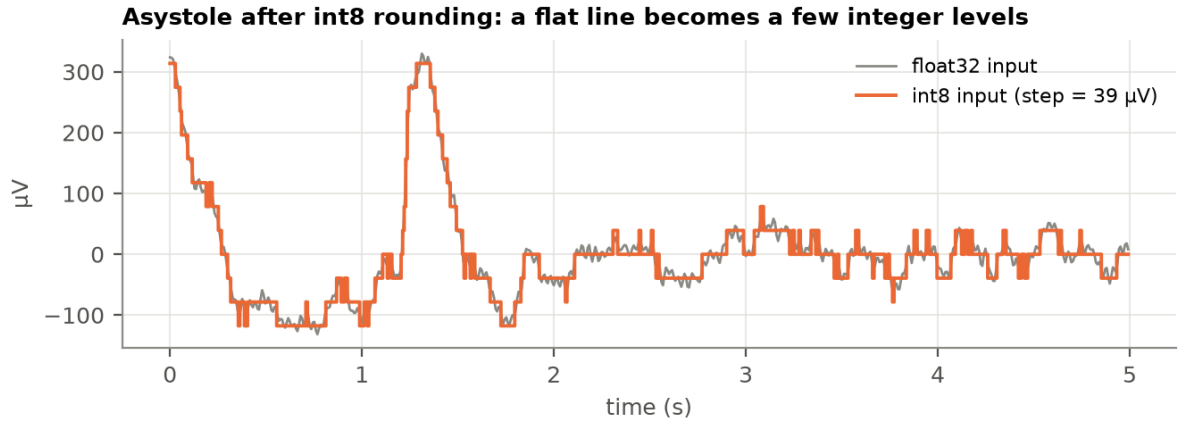


Fig. 9: A typical asystole clip before (grey) and after (orange) int8 rounding of the input. A few integer levels must carry the whole signal.

6.7 A threshold tuned on too few patients

We also chose a threshold per fold on the validation patients (Section 5.5). It did not help. For 4 of the 9 models in Table 4 it passed fewer rows than plain 0.5, and for none of them did it pass more. The reason is size: each validation set held only 13 patients. Chasing the strict 99% NSR goal on so few people pushed the threshold up to 0.8–0.95 in several folds, which then cost coarse-VF sensitivity on the test patients. Fold 0 shows the other side of the problem: its validation patients had no asystole and only 7 rapid-VT clips, and NSR scored 100% at every threshold, so every threshold from 0.05 to about 0.9 tied and the tie-break returned 0.5. Because both thresholds were planned before the results were seen, we report both (tables/aha_by_model.csv), and we use 0.5 as the headline because it held up better on unseen patients.

7. Discussion

So, back to the two questions.

How far can the model be compressed? From 1,975 KiB to 93 KiB: 35% of the filters, int8 throughout. At that size it fits the STM32L432 with room to spare (10 KiB of the 64 KiB RAM) and meets every AHA target our data can test properly. Quantisation did the first four-times saving almost for free. Structured pruning did the rest, but only with fine-tuning, and only because it removes whole filters; zeroing individual weights saved nothing on the chip.

Compress big or train small? At the same shape, training from scratch was as good as pruning, and a little better on shockable rhythms. The large model was still useful: training and pruning it is how we found a shape that works. But for a real device, the cheaper path is to search for the right small architecture and train it directly.

These answers come with limits, and it would be easy to state the results in a way that sounds better than they are. A model that "fits the chip and meets 4 of 5 AHA targets" sounds close to ready. Here is what that sentence leaves out.

- **Asystole is effectively one patient.** 113 of its 147 clips come from a single VFDB recording (vfdb/427). No model, compressed or not, passes the asystole row, and with only 6 patients, one badly classified person drags the lower confidence limit far below the 92% minimum. Our asystole labels also come from the database annotations, so they include low-amplitude activity up to

several hundred µV, unlike Kerber's strict < 100 µV definition. Any claim about asystole needs far more data.

- **Pass and fail sit on a knife edge.** Several rows pass or fail by less than one percentage point (NSR: 98.6% vs a 99% goal). The 35%-width scratch model passes 3 rows in float32 and 4 in int8, not because rounding helped, but because NSR moved from just below the line to just above it. We trained each model once per fold; a different random seed could move a row either way. The safe reading is that the 35%-width models are *as good as* the uncompressed model, not better.
- **The data is not what a real AED sees.** These are clean Holter and monitoring recordings. Real AED signals come through pads on a moving, often wet chest, usually during CPR, which adds large artefacts. No such recordings are publicly available.
- **Two labelling choices are ours.** Rapid VT means ≥ 180 bpm here; the AHA does not fix a rate. Fine VF has only 2 clips, so the AHA's "report only" row is effectively empty.
- **The chip numbers are estimates.** We did not run the model on a physical board. Flash is the model file only (the TensorFlow Lite Micro runtime adds its own code), RAM is a lower bound, and latency is calculated from operation counts, not measured.

None of these undo the main findings, which compare models trained on the same data and judged the same way. But they decide what the findings can be used for: evidence about compression, not evidence that this model is safe to put in a defibrillator.

The asystole result also points to a concrete next step. The int8 input step is set by the loudest calibration clips, which leaves quiet signals with only a few levels. Scaling the input so its int8 range matches the amplitudes that matter, or using quantisation-aware training, could give asystole more resolution at no extra cost in size. Together with running the model on a real STM32 board and training several seeds per configuration, that is where this work would go next.

8. Conclusion

Back at the void deck, the AED on the wall has about ten seconds and a chip smaller than a phone photo. This project showed that a deep-learning shock-advice model can live on such a chip: shrunk 21 times, from 1,975 KiB to 93 KiB, by keeping 35% of its filters and storing everything as 8-bit integers. It still meets the AHA targets for coarse VF, rapid VT, normal rhythm and other non-shockable rhythms, judged on patients it never saw. A network of the same shape trained from scratch did just as well, so the big model's real contribution was showing us which small design to build.

The more lasting lesson is about how compression is measured. A model can keep its overall accuracy while one rhythm quietly falls apart, as asystole did in the 50% pruned model. For a device whose mistakes are paid for in shocks, compression has to be judged rhythm by rhythm, patient by patient, and reported in full, including the rows that fail.

References

David, R., Duke, J., Jain, A., et al. (2021). TensorFlow Lite Micro: Embedded machine learning for TinyML systems. *Proceedings of Machine Learning and Systems*, 3.

Goldberger, A. L., Amaral, L. A. N., Glass, L., et al. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23), e215–e220.

Han, S., Pool, J., Tran, J., & Dally, W. J. (2015). Learning both weights and connections for efficient neural networks. *Advances in Neural Information Processing Systems 28*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Jacob, B., Kligys, S., Chen, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Jaureguibeitia, X., Zubia, G., Irusta, U., et al. (2020). Shock decision algorithms for automated external defibrillators based on convolutional networks. *IEEE Access*, 8.

Jia, Z., et al. (2023). Life-threatening ventricular arrhythmia detection challenge in implantable cardioverter–defibrillators. arXiv:2305.05105.

Kerber, R. E., Becker, L. B., Bourland, J. D., et al. (1997). Automatic external defibrillators for public access defibrillation: Recommendations for specifying and reporting arrhythmia analysis algorithm performance, incorporating new waveforms, and enhancing safety. *Circulation*, 95(6), 1677–1682.

Krishnamoorthi, R. (2018). Quantizing deep convolutional networks for efficient inference: A whitepaper. arXiv:1806.08342.

Lai, L., Suda, N., & Chandra, V. (2018). CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs. arXiv:1801.06601.

Li, H., Kadav, A., Durdanovic, I., Samet, H., & Graf, H. P. (2017). Pruning filters for efficient ConvNets. *International Conference on Learning Representations (ICLR)*.

Liu, Z., Sun, M., Zhou, T., Huang, G., & Darrell, T. (2019). Rethinking the value of network pruning. *International Conference on Learning Representations (ICLR)*.

Moody, G. B., & Mark, R. G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45–50.

Nguyen, M. T., Nguyen, B. V., & Kim, K. (2018). Deep feature learning for sudden cardiac arrest detection in automated external defibrillators. *Scientific Reports*, 8, 17196.

US Food and Drug Administration (2022). Premarket approval P210015: Avive Connect AED System.

Appendix: Code and reproduction

All code, results and this report are in the project repository. `code/data.py` downloads and prepares the data, `code/run_experiments.py --all` trains and compresses every model, `code/summarise.py` and `code/make_figures.py` produce every table and figure in this report, and `code/demo.py` runs the final int8 model on any ECG clip. The README gives step-by-step instructions.